

Low-Latency Execution Infrastructure for Funding-Rate Arbitrage: Architecture and a Reproducible Single-Host Benchmark

Kun Xue
Godzilla Foundation

Abstract

Funding-rate arbitrage combines a perpetual-futures position with an offsetting spot or derivatives position to receive funding while limiting directional exposure. Although the economic signal is relatively transparent, implementation remains sensitive to stale market data, delayed order submission, incomplete hedges, and venue-specific failures. This paper presents `godzilla.dev`, an open-source, self-hosted C++/Python execution framework for funding-rate arbitrage, market making, and other latency-sensitive cryptocurrency strategies. The framework separates market-data, strategy, and trade-execution responsibilities into independently managed processes connected through a journal-based shared-memory event path. We describe the process model, persistent event records, the native C++ and Python strategy boundary, and the operational rationale for process isolation and replayable state.

We evaluate a narrowly defined local software path using native mock market-data and trade components on a single host. The benchmark measures the interval from generation of a synthetic top-of-book journal frame to generation of a corresponding local order-report frame. It excludes network transport, exchange protocol handling, matching-engine processing, fills, queue position, and strategy profitability. On an Intel Core i7-1360P host, five runs retained 900 post-warm-up order cycles each. Run-level median latency ranged from 121.1 to 135.4 microseconds, while run-level p99 latency ranged from 434.8 to 662.0 microseconds. These measurements characterize only the specified single-host, low-utilization configuration and do not establish a production latency bound or a comparison with another trading framework. The benchmark scripts, journal-derived event records, analysis code, and per-run summaries are published with the source repository.

1 Introduction

Perpetual futures use periodic funding transfers to help keep contract prices near their underlying reference markets [1]. A funding-rate arbitrage strategy typically offsets a perpetual position with spot or another derivative and attempts to receive funding while limiting directional exposure. The signal is transparent, but implementation is not: both legs must be established, maintained, and unwound while prices, available liquidity, funding expectations, and venue conditions change.

Execution infrastructure therefore matters even when the strategy is not an ultra-short-horizon alpha strategy. A delayed order can become stale, a delayed hedge can create unintended exposure, and a slow response during basis compression can consume the expected funding income. Internet latency and exchange matching are important, but the software path controlled by the operator also contributes delay and variability through market-data processing, inter-process transfer, strategy dispatch, order construction, and execution routing.

Open-source trading systems optimize for different objectives. Connector-rich bot frameworks emphasize venue coverage, strategy templates, and ease of use. Proprietary low-latency stacks emphasize deterministic execution, but are expensive to build and difficult to audit. `godzilla.dev` targets the space between them: an open-source, self-hosted C++/Python execution framework for cryptocurrency strategies that require process isolation, persistent event records, and a short native execution path.

This paper makes four contributions:

1. It documents a process-isolated market-data, strategy, and trade architecture connected through a journal-based shared-memory path.
2. It identifies the provenance of the framework components and explains how native execution components coexist with Python strategy interfaces without requiring Python in the measured callback path.
3. It defines a reproducible journal-only benchmark for the local path from synthetic depth generation to a local order report, with timestamps extracted after execution from the framework’s event records.
4. It reports repeated-run latency distributions and explicitly separates the measured software path from network, exchange, fill, reliability, and profitability claims.

The title deliberately refers to funding-rate arbitrage as the application domain rather than claiming that the experiment measures cross-exchange execution. The evaluated path contains one synthetic market-data source and one native mock trade process. Cross-venue coordination remains outside the present experiment.

2 Background and Related Work

2.1 Funding-rate arbitrage and execution risk

For a hedged notional N and realized funding rate f , the funding payment can be represented as

$$\Pi_{\text{funding}} = Nf. \quad (1)$$

A simplified net-profit decomposition is

$$\begin{aligned} \Pi_{\text{net}} = & \Pi_{\text{funding}} + \Pi_{\text{basis}} - C_{\text{fees}} - C_{\text{slippage}} \\ & - C_{\text{capital}} - C_{\text{execution}}. \end{aligned} \quad (2)$$

This decomposition is not a pricing model; it is an operator-side accounting identity that highlights why a nominally delta-neutral trade remains execution-sensitive. The system must coordinate legs and manage stale prices, rejected orders, partial fills, inventory drift, and venue degradation. Low internal latency cannot guarantee profitability, but excessive or variable internal latency increases the interval during which the intended hedge and the actual portfolio differ.

He et al. derive no-arbitrage prices for perpetual futures in frictionless markets and bounds under trading costs [1]. Makarov and Schoar document persistent cross-exchange cryptocurrency price deviations and relate their persistence to market segmentation and capital-mobility frictions [2]. These results motivate the application domain but do not imply that a local software microbenchmark is itself an arbitrage-performance evaluation.

2.2 Trading frameworks and low-latency event paths

Hummingbot is an open-source Python framework with broad centralized- and decentralized-exchange connectivity and ready-made strategy workflows [3]. Those are distinct design strengths. The present paper does not claim a cross-framework performance result. A fair comparison would require pinned versions, equivalent strategy semantics, identical input streams and mock endpoints, disclosed configurations, and review of both setups.

The market-design literature documents the economic stakes of latency competition [4]. Engineering practice for latency-sensitive systems emphasizes bounded hot-path work, pre-allocation, cache locality, clock discipline, and explicit treatment of tail latency. The LMAX Disruptor is a published example of a pre-allocated event structure designed to reduce queueing

and allocation overhead [5]. The journal substrate used by godzilla.dev is not asserted to be equivalent to the Disruptor, but it belongs to the broader family of append-oriented event transports intended to support low-overhead handoff and replay.

3 Framework Architecture

3.1 Component provenance

godzilla.dev builds on the Kungfu journal substrate and runtime concepts [7]. The benchmark-specific connectors, strategy, launch scripts, journal analyzer, aggregation scripts, and result artifact are distributed in the godzilla.dev community repository [8]. Table 1 separates reused infrastructure from the components evaluated in this paper.

Table 1: Component provenance and role in the evaluated artifact.

Component	Provenance	Role in this paper
Journal substrate and core runtime concepts	Kungfu-derived code included in the source snapshot	Inter-process transport, frame timestamps, and persistent event records
Cryptocurrency connectors and deployment layer	godzilla.dev	Application-facing integration; not exercised by the native mock benchmark
Native mock market data and trade components	godzilla.dev	Deterministic local source and local order-report endpoint
Native benchmark strategy	godzilla.dev	Minimal C++ callback that converts an accepted depth event into one order request
Journal analyzer and multi-run aggregation	godzilla.dev	Post-run event matching, latency decomposition, and run-level summaries

This paper does not claim that every journal operation is lock-free or wait-free. Establishing such properties would require a separate implementation audit and formal argument.

3.2 Process model

The runtime separates market-data (**md**), strategy, and trade (**td**) responsibilities into different processes (Figure 1). The market-data process converts venue-specific input into normalized events. The strategy consumes those events and emits normalized order requests. The trade process translates requests into venue actions and publishes order-state events. Master and ledger services provide coordination and state recording.

Process separation is operational as well as architectural. Venue adapters can be restarted without embedding exchange-specific code in each strategy process, and strategy failures do not have to corrupt market-data parsing or trade connectivity. Persistent event records also provide a common source for post-run inspection and local state reconstruction, subject to journal completeness and reconciliation with venue state.

3.3 Journal-based shared-memory path

Processes exchange typed event frames through the journal substrate. Relevant frames carry generation timestamps, trigger timestamps, and source and destination identities. Consumers subscribe to the journal locations they require, while the recorded stream remains available for later inspection.

The same mechanism serves two roles in the benchmark. During execution it transports events between processes. After execution it supplies timestamps for depth, order-input, and order-report frames. Synchronous CSV tracing is disabled in the measured path, and latency

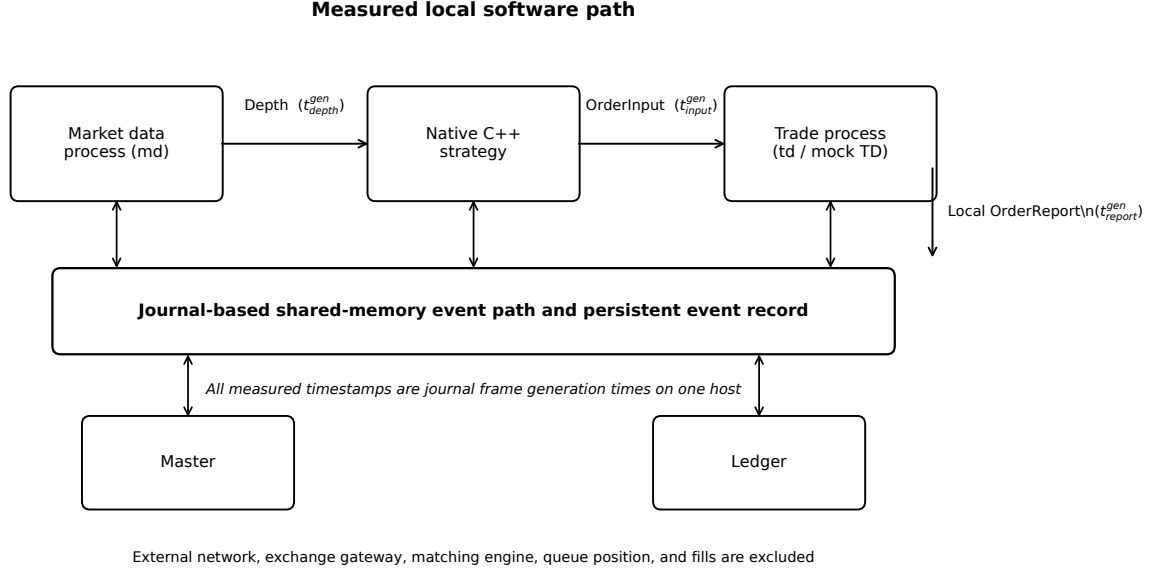


Figure 1: Process model and measured local event path. The benchmark replaces external venue interfaces with native mocks. Journal frame generation times define the measured intervals. Network transport, exchange gateways, matching, queue position, and fills are excluded.

is reconstructed from journal records after the run. This avoids adding text formatting and trace-file writes to the path under measurement.

The term *persistent event record* is used instead of a stronger storage-durability guarantee. The experiment does not test persistence across storage-device failure, operating-system crash, or incomplete journal flush.

3.4 C++/Python boundary

The framework exposes strategy lifecycle and market events through pybind11 [6], allowing Python strategies to use the native runtime while retaining rapid iteration. Strategies can also implement the same interface in C++ when measurements show that callback execution lies on a latency-critical path.

The strategy evaluated here is native C++. Its depth callback alternates between buy and sell, selects the corresponding displayed best price, constructs one limit order, and calls `insert_order`. Python remains part of process bootstrap and module loading, but it is not used to execute the measured callback logic. A Python-path benchmark would answer a different and practically important question and is not mixed with the native result.

3.5 Operational design rationale

The architecture is informed by production requirements in self-hosted cryptocurrency trading systems, but production deployment is not used as empirical evidence for the latency result. Four requirements motivate the design.

First, a persistent event history makes recovery and post-incident analysis operate on the same records used by the runtime. Second, isolating venue adapters allows partial API degradation to be contained and a connector to be restarted without stopping unrelated processes. Third, journal timestamp ordering makes clock stability a first-class operational concern. The single-host topology used here places the measured path inside one system clock domain. Fourth, a safe

restart must reconcile journal-derived local state with venue open orders and positions before strategy activity resumes.

These are design requirements, not evaluated reliability claims. Failure-injection tests, recovery-time measurements, and long-duration production statistics remain future work.

4 Benchmark Methodology

4.1 Research question and scope

The experiment asks:

Under a controlled, single-host, journal-only configuration, how long does the framework take to transform a synthetic top-of-book event into a locally generated order report?

The experiment is a software-path microbenchmark. It does not measure strategy profitability, funding capture, fill probability, queue position, market impact, exchange matching, gateway latency, Internet or colocation network latency, connector breadth, usability, or long-duration reliability.

4.2 Measured event path

The measured sequence is

$$\text{Depth}_{md} \rightarrow \text{Strategy} \rightarrow \text{OrderInput}_{strategy} \rightarrow \text{TD} \rightarrow \text{OrderReport}_{td}. \quad (3)$$

For a matched event cycle, the analyzer computes

$$T_{\text{depth} \rightarrow \text{input}} = t_{\text{input}}^{\text{gen}} - t_{\text{depth}}^{\text{gen}}, \quad (4)$$

$$T_{\text{input} \rightarrow \text{report}} = t_{\text{report}}^{\text{gen}} - t_{\text{input}}^{\text{gen}}, \quad (5)$$

$$T_{\text{depth} \rightarrow \text{report}} = t_{\text{report}}^{\text{gen}} - t_{\text{depth}}^{\text{gen}}. \quad (6)$$

The primary metric is called *depth-to-local-order-report latency*. It should not be interpreted as exchange-acknowledged order latency or fill latency. The first stage includes journal delivery, callback dispatch, side and price selection, order construction, and publication of the order-input frame. The second includes trade-process consumption and generation of a local mock order report.

4.3 Event matching

Order-input frames are joined to order reports by `order_id`. Depth frames are joined to order-input frames using the nearest unused prior frame with matching symbol, side, and price; a time-based fallback exists in the analyzer. In the five reported runs, all 1,000 cycles per run were resolved by the primary matching rules, and no fallback or unmatched order was reported.

The depth matching key is not a propagated causal identifier. Matching success therefore means that the analyzer resolved every cycle under this deterministic, single-symbol workload; it does not prove uniqueness under repeated prices, bursts, multiple symbols, or backlog. Propagating a source depth event identifier through `OrderInput` and `OrderReport` is required for a stronger causal join.

Table 2: Captured benchmark configuration.

Item	Configuration
CPU	Intel Core i7-1360P, 12 cores / 16 logical CPUs, 18 MiB L3
Operating system	Linux 6.14.0-37-generic, x86-64
Runtime topology	Master, ledger, native mock MD, native mock TD, native C++ strategy
Logical CPU affinity	Master=0, ledger=1, MD=2, TD=3, strategy=4
Market-data interval	300,000 ns
MD wait policy	Sleep followed by a 100,000 ns spin window
Trace mode	Journal only; benchmark CSV tracing disabled
Input per run	5,000 depth events
Orders per run	1,000
Warm-up exclusion	First 100 matched cycles
Retained observations	900 per run; 4,500 total
Independent repetitions	5

4.4 Experimental configuration

The working tree was based on source commit `5b9cee052591fc77b03925fbe8500632ebc8eaaa`. The published result artifact is identified by commit `da9dd09839c6ce16ab73b1a7a11ae1b5ed4e9349` [8]. Table 2 reports the captured configuration.

The benchmark command used the 100,000 ns spin setting recorded in the result artifact and repository benchmark README. A separate methodology document in the same historical snapshot listed 50,000 ns; that documentation inconsistency does not change the stored result files, but it should be corrected in the repository before a new measurement campaign.

The exact compiler version, optimization flags, CPU frequency governor, turbo and C-state configuration, memory configuration, hybrid-core mapping, simultaneous-multithreading sibling placement, interrupt affinity, clock-synchronization status, and background workload were not captured in the published result artifact. The present result must therefore be interpreted as a measured baseline for the stated host snapshot, not as a portable hardware or operating-system performance claim.

4.5 Statistical reporting

Each run retained 900 observations after warm-up removal. We report run-specific percentiles and summarize their minimum, median, mean, and maximum across the five runs. The arithmetic mean of all latency observations is not used as the headline metric because the distributions are asymmetric and tail events are operationally important.

The artifact also reports p99.9. With only 900 retained observations per run, a p99.9 estimate is determined almost entirely by the largest observations and is not interpreted as a stable tail statistic in this paper. Extreme-tail claims require substantially more observations and more independent repetitions.

5 Results

5.1 Repeated-run latency

Table 3 summarizes the run-level depth-to-local-order-report percentiles. Figure 2 shows each run separately.

The mean of the five run-level arithmetic means was 161.5 μ s. Run-level maxima ranged from 719.9 to 1,450.0 μ s. The result supports a narrow statement: in the specified configuration, the

Table 3: Run-level depth-to-local-order-report latency in microseconds. The aggregation unit is the benchmark run.

Percentile	Minimum	Median	Mean	Maximum
p50	121.1	121.3	124.9	135.4
p90	256.0	281.4	278.0	298.6
p99	434.8	447.3	485.9	662.0

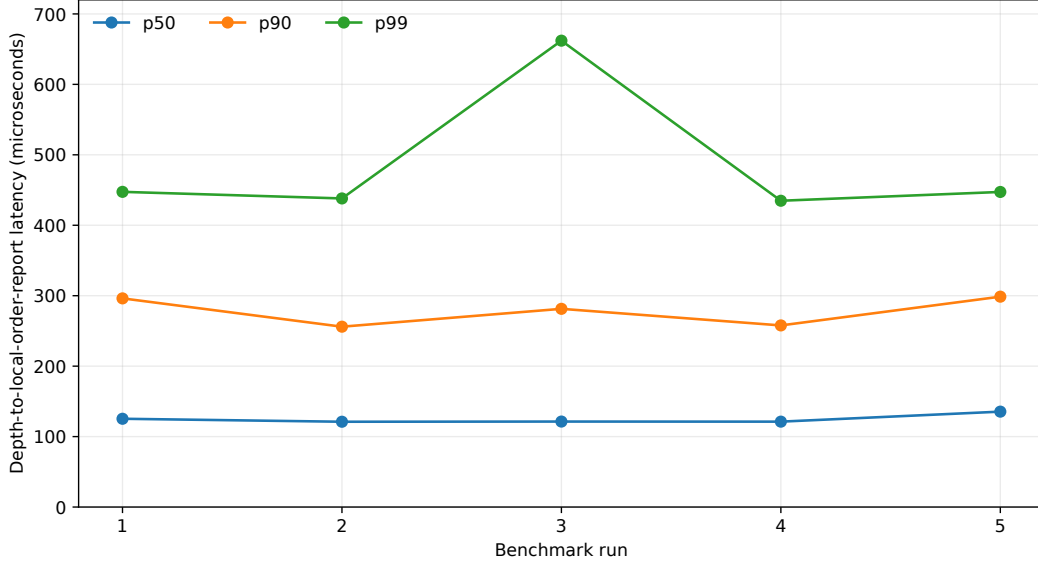


Figure 2: Per-run latency percentiles. Run 3 contains the largest p99 excursion, illustrating why a single best run would overstate predictability.

median local cycle completed in approximately 121–135 μ s, while p99 varied more substantially across runs.

5.2 Stage decomposition

Table 4 separates the market-data-to-order-input and order-input-to-local-report stages. The table reports medians and means of the five run-specific percentiles.

Table 4: Stage latency in microseconds. Stage-specific percentiles are marginal quantiles and are not additive.

Stage	p50		p90		p99	
	Median	Mean	Median	Mean	Median	Mean
Depth to order input	99.8	100.0	147.2	147.6	336.2	335.6
Order input to local report	22.3	26.3	131.4	131.5	187.7	206.5
End-to-end local path	121.3	124.9	281.4	278.0	447.3	485.9

The median is dominated by the depth-to-order-input stage. At higher percentiles, both stages contribute variability. Run 3 contained a 1.334 ms maximum in the order-input-to-report stage and the largest end-to-end p99. The current instrumentation cannot attribute that excursion to a particular scheduler, cache, wake-up, or runtime mechanism.

5.3 Interpretation

The 300 μ s input interval is a workload parameter, not a performance target. It corresponds to approximately 3,333 synthetic depth events per second, and the strategy emits only the first 1,000 orders from a 5,000-event stream. Because the median measured path is shorter than the input interval, this experiment principally characterizes low-utilization latency rather than sustained saturation behavior.

The experiment does not support a universal latency bound, a production service-level objective, or superiority over another framework. It also does not evaluate coordinated two-leg submission, hedge completion, or cross-venue timing. Its value is narrower: it defines auditable event boundaries and preserves per-event records for a repeatable local software-path measurement.

6 Threats to Validity and Limitations

Sample size and tail estimation. Five runs and 4,500 retained observations provide an initial engineering baseline but are inadequate for stable extreme-percentile estimation. A stronger study should predeclare at least 30 independent measured repetitions, use substantially longer runs, and report confidence intervals across runs.

Event causality. Depth-to-order-input matching relies on symbol, side, price, and temporal ordering rather than a propagated causal event identifier. The deterministic workload reduces ambiguity, but it does not eliminate the methodological weakness. A source event identifier should be carried through the order lifecycle before burst and multi-symbol experiments.

Workload representativeness. Synthetic constant-rate top-of-book messages do not reproduce clustered arrivals, message-size variation, exchange-specific decoding, burst backlogs, or order-book reconstruction. Recorded market-data replay and controlled burst datasets should be added.

Short duration. The finite workload is too short to characterize thermal behavior, periodic kernel activity, journal rollover, long-lived memory pressure, or recovery after faults. Future experiments should include sustained runs and report throughput, queue depth, dropped events, CPU utilization, and deadline-miss ratios.

Missing environment controls. The artifact does not capture several configuration variables that can materially affect latency, including compiler flags, CPU governor, hybrid-core mapping, C-states, and background load. Results on other hardware or a tuned server may be higher or lower; no direction is inferred from the present experiment.

Native-only callback path. The benchmark uses a native C++ strategy and native mock components. A Python callback result should be reported separately rather than blended with the native result.

No external exchange path. Network sockets, exchange protocols, gateways, matching engines, acknowledgments, fills, and queue position are intentionally excluded. A testnet or controlled gateway experiment is required to characterize deployed end-to-end behavior.

No comparative evaluation. The paper does not compare godzilla.dev with Hummingbot or another system. Such a study would need version-pinned implementations, identical workloads and semantics, disclosed tuning, and independent review. Connector breadth, operability, and ecosystem maturity should be evaluated separately from latency.

Reliability and scaling. Failure recovery, reconciliation time, multi-host clocks, container overhead, connector breadth, and long-duration reliability are not evaluated. The single-host shared-memory design favors local communication and process isolation but does not establish a multi-host scaling result.

7 Conclusion

This paper described a self-hosted C++/Python execution framework for funding-rate arbitrage and other latency-sensitive cryptocurrency strategies. The architecture separates market data, strategy, and trade execution into local processes connected by a journal-based shared-memory event path. The benchmark measured a precisely bounded local interval from synthetic depth-frame generation to a native mock trade-process order report.

Across five runs on an Intel Core i7-1360P host, run-level median latency ranged from 121.1 to 135.4 μ s, and run-level p99 ranged from 434.8 to 662.0 μ s. These values characterize one short, low-utilization, single-host configuration. They do not measure exchange latency, cross-venue coordination, fills, production reliability, or profitability.

The principal contribution is methodological as much as numerical: local latency claims should define event boundaries, remove synchronous tracing from the measured path, preserve auditable per-event records, report run-to-run tail variability, and state what is excluded. The published artifact provides a baseline for longer workloads, strict causal identifiers, burst and replay datasets, Python-path characterization, and independently reviewable comparative experiments.

Artifact Availability

The source snapshot, benchmark launch scripts, journal analyzer, multi-run aggregation, plotting tools, and stored result artifacts are available in the godzilla.dev community repository at artifact commit `da9dd09839c6ce16ab73b1a7a11ae1b5ed4e9349` [8]. The reported measurements are stored under `scripts/benchmark/analysis/spin_100000_confirm`. A future revision should archive the artifact under an immutable release or DOI and publish checksums for raw result files.

Competing Interests

The author is affiliated with the Godzilla Foundation and is a developer of the evaluated system. No external framework is evaluated in this paper.

References

- [1] S. He, A. Manela, O. Ross, and V. von Wachter. Fundamentals of perpetual futures. *arXiv preprint arXiv:2212.06888*, 2022, revised 2024.
- [2] I. Makarov and A. Schoar. Trading and arbitrage in cryptocurrency markets. *Journal of Financial Economics*, 135(2):293–319, 2020. doi:10.1016/j.jfineco.2019.07.001.

- [3] Hummingbot Foundation. Hummingbot documentation and source repository. <https://hummingbot.org/docs/> and <https://github.com/hummingbot/hummingbot>, accessed July 2026.
- [4] E. Budish, P. Cramton, and J. Shim. The high-frequency trading arms race: Frequent batch auctions as a market design response. *The Quarterly Journal of Economics*, 130(4):1547–1621, 2015. doi:10.1093/qje/qjv027.
- [5] M. Thompson, D. Farley, M. Barker, P. Gee, and A. Stewart. Disruptor: High performance alternative to bounded queues for exchanging data between concurrent threads. LMAX Exchange technical paper, May 2011. <https://lmax-exchange.github.io/disruptor/files/Disruptor-1.0.pdf>.
- [6] W. Jakob, J. Rhineland, and D. Moldovan. pybind11: Seamless operability between C++11 and Python. <https://github.com/pybind/pybind11>, 2017.
- [7] Kungfu Origin. Kungfu Trader. <https://github.com/kungfu-origin/kungfu>, Apache License 2.0, accessed July 2026.
- [8] Godzilla Foundation. godzilla.dev community repository and benchmark artifact. Artifact commit da9dd09839c6ce16ab73b1a7a11ae1b5ed4e9349, 2026. <https://github.com/godzilla-foundation/godzilla-community/tree/da9dd09839c6ce16ab73b1a7a11ae1b5ed4e9349>.